Documentación Técnica: Módulo de Interfaz y Pruebas

Esta sección detalla los módulos implementados en `app.js` para la gestión de la red bayesiana, abarcando la exportación visual, el control de estados y la manipulación segura de la estructura de nodos.

1. Exportación de Grafo a PNG

Permite al usuario descargar el estado actual del diagrama visual en una imagen de alta resolución con fondo sólido.

JavaScript

```
C/C++
$('btn-exportar-img').addEventListener('click', () => {
    // cy.png() toma la foto directamente del lienzo con fondo blanco
    const pngUrl = cy.png({ bg: '#ffffff', full: true });

    // Crear un enlace invisible para forzar la descarga nativa
    const link = document.createElement('a');
    link.href = pngUrl;
    link.download = 'grafo-red-bayesiana.png';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
});
```

- **¿Qué hace?** Genera un archivo `.png` del lienzo de Cytoscape y lo descarga al equipo del usuario.
- **¿Cómo lo hace?** Utiliza la API nativa de Cytoscape (`cy.png`) para renderizar el estado actual del canvas. Como los navegadores no permiten forzar descargas directamente por seguridad, se crea un elemento `<a>` temporal, se le asigna la imagen como fuente en formato base64 (`href`), se simula un clic programático (`link.click()`) y se destruye el elemento para no ensuciar el DOM.

2. Sistema de Historial (Deshacer / Rehacer)

Implementa una máquina del tiempo en memoria para revertir o recuperar acciones estructurales en el lienzo mediante atajos de teclado estándar.

JavaScript

```C/C++
let historialRed = [];
let pasoActual = -1;

function guardarEstadoRed() {
    const estado = $('editor').value;
    if (pasoActual === -1 || historialRed[pasoActual] !== estado) {
        historialRed = historialRed.slice(0, pasoActual + 1);
        historialRed.push(estado);
        pasoActual++;
    }
}

// Guardar el estado inicial apenas cargue la página y al soltar el ratón
setTimeout(guardarEstadoRed, 500);
document.addEventListener('mouseup', () => {
    setTimeout(guardarEstadoRed, 100);
});
$('btn-aplicar').addEventListener('click', guardarEstadoRed);

// Detectar atajos de teclado (Ctrl+Z / Ctrl+Y)
document.addEventListener('keydown', (e) => {
    if (e.ctrlKey && e.key.toLowerCase() === 'z') {
        e.preventDefault();
        if (pasoActual > 0) {
            pasoActual--;
            loadText(historialRed[pasoActual]);
        }
    }
    if (e.ctrlKey && e.key.toLowerCase() === 'y') {
        e.preventDefault();
        if (pasoActual < historialRed.length - 1) {
            pasoActual++;
            loadText(historialRed[pasoActual]);
        }
    }
});
```

- **¿Qué hace?** Permite usar `Ctrl+Z` para deshacer y `Ctrl+Y` para rehacer cualquier cambio en la red bayesiana.
- **¿Cómo lo hace?** Mantiene un arreglo (`historialRed`) que guarda copias exactas del JSON (que es la "verdad absoluta" de la red) cada vez que ocurre una acción significativa (clics, botón aplicar). La variable `pasoActual` actúa como un puntero. Al presionar `Ctrl+Z`, el puntero retrocede una posición y manda ese texto JSON a la función `loadText()`, la cual redibuja el lienzo. Si se hace un cambio nuevo estando atrás en el historial, la función `slice` recorta el futuro alternativo para evitar conflictos de línea temporal.

3. Generación Dinámica de Tablas de Probabilidad (CPT)

Calcula el producto cartesiano de los estados de los nodos padres para construir una tabla matemática estrictamente válida para el motor bayesiano.

JavaScript

```
C/C++
function buildDefaultCPT(node, allNodes) {
       const parentNodes = node.parents.map(pId => allNodes.find(n => n.id === pId)).filter(Boolean);
    let combinations = [{}];

    // Producto cartesiano de los estados de todos los padres
    for (const p of parentNodes) {
        const newCombos = [];
        for (const c of combinations) {
            for (const state of p.states) {
                newCombos.push({ ...c, [p.id]: state });
            }
        }
        combinations = newCombos;
    }

    // Probabilidad equitativa por defecto
    const prob = Number((1 / node.states.length).toFixed(4));
    const probs = {};
    node.states.forEach(s => probs[s] = prob);

    return combinations.map(c => ({
        given: c,
        probs: { ...probs }
    }));
}
```

- **¿Qué hace?** Genera la estructura exacta de objetos `cpt: [{given: {}, probs: {}}]` que exige el validador estricto de la aplicación cada vez que se crea o edita un nodo.
- **¿Cómo lo hace?** Itera sobre los nodos padres para multiplicar sus estados (ej. si el padre tiene 2 estados y el hijo 2, genera 4 filas). Asigna a todas las combinaciones una probabilidad equitativa inicial (`1 / número de estados`). Esto previene la excepción de "fila no es un objeto" y evita la destrucción de las aristas en el grafo.

4. Delegación de Eventos para Formularios de Nodos (Crear/Editar/Eliminar)

Gestiona la interfaz de los paneles inferiores asegurando sincronización total entre el lienzo, el JSON subyacente y las dependencias de parentesco.

JavaScript

```
C/C++
let nodoSeleccionadoId = null;

// Cargar datos automáticamente al tocar el lienzo
document.addEventListener('mouseup', () => {
    setTimeout(() => {
        try {
            if (typeof cy !== 'undefined' && cy.$) {
                const sel = cy.$('node:selected');
                if (sel.length > 0) {
                    const id = sel[0].id();
                    const elEditor = document.getElementById('editor');
                    if (elEditor && elEditor.value) {
                        const data = JSON.parse(elEditor.value);
                        const nodo = data.nodes.find(n => n.id === id);
                        if (nodo) {
                            nodoSeleccionadoId = id;
                            document.getElementById('nodo-id').value = id;
                                    document.getElementById('nodo-estados').value =
nodo.states.join(', ');
                        }
                    }
                }
            }
        } catch(e) {}
    }, 150);
});

// Radar global para botones Guardar y Eliminar
document.addEventListener('click', (e) => {
    if (e.target && e.target.id === 'btn-guardar-nodo') {
```

```
        // ... (extracción de valores del DOM) ...
        const data = JSON.parse(elEditor.value);
        let idViejo = nodoSeleccionadoId || 'Aspersor';
        let nodeIndex = data.nodes.findIndex(n => n.id === idViejo);

        if (nodeIndex !== -1 && idViejo !== nuevoId) {
            data.nodes[nodeIndex].id = nuevoId;
            data.nodes.forEach(n => {
                    if (n.parents) n.parents = n.parents.map(p => p === idViejo ?
nuevoId : p);
            });
            // Migrar posiciones
        } else if (nodeIndex === -1) {
              data.nodes.push({ id: nuevoId, type: "discrete", states: [], parents:
[], cpt: [] });
            nodeIndex = data.nodes.length - 1;
        }

        let targetNode = data.nodes[nodeIndex];
        targetNode.states = estados;
        targetNode.cpt = buildDefaultCPT(targetNode, data.nodes);

        // Reconstruir CPT de TODOS los nodos hijos
        data.nodes.forEach(n => {
            if (n.parents && n.parents.includes(nuevoId)) {
                n.cpt = buildDefaultCPT(n, data.nodes);
            }
        });

        elEditor.value = JSON.stringify(data, null, 2);
        nodoSeleccionadoId = nuevoId;
        if (typeof loadText === 'function') loadText(elEditor.value);
    }

    if (e.target && e.target.id === 'btn-eliminar-nodo') {
        // ... (búsqueda del nodo por ID) ...
        if (nodeIndex !== -1) {
            data.nodes.splice(nodeIndex, 1);
            data.nodes.forEach(n => {
                if (n.parents && n.parents.includes(id)) {
                    n.parents = n.parents.filter(p => p !== id);
                     n.cpt = buildDefaultCPT(n, data.nodes); // Reconstruir tabla sin
ese padre
                }
            });
            // ... (Actualización de DOM y JSON) ...
        }
    }
});
```

- **¿Qué hace?** Autocompleta los campos al seleccionar un nodo. Permite renombrar o crear variables y eliminarlas sin dejar conexiones huérfanas o errores estructurales.
- **¿Cómo lo hace?**

  Autocompletado: Un retraso `setTimeout` de 150ms espera a que Cytoscape registre la selección nativa, extrae el ID, busca el objeto en el JSON y lo inyecta en los `input`.

  Delegación global: En lugar de asignar un `.addEventListener` directo a los botones (que pueden perder la referencia si el DOM se recarga), el `document.addEventListener('click')` intercepta todos los clics de la página y filtra por la ID del objetivo (el "radar de clics").

  Migración segura: Al renombrar, escanea el arreglo `parents` de todos los demás nodos y actualiza el nombre viejo por el nuevo. Al eliminar, borra las referencias del arreglo `parents` en los nodos dependientes y vuelve a ejecutar `buildDefaultCPT` en cada uno para sanear las tablas de probabilidad al instante.